Application of large language models (LLM) to synthesize and interpret heterogeneous market data for personalized investment decision support

Motivation

Our goal is to create a system that supports investment decision-making by transforming large volumes of heterogeneous financial data (e.g., reports and market data) into personalized, actionable insights. Existing tools provide access to data but require significant manual interpretation and lack alignment with individual investor strategies. We aim to leverage Large Language Models (LLMs) combined with Retrieval-Augmented Generation (RAG) to synthesize diverse data sources and generate context-aware interpretations tailored to factors such as risk tolerance and investment horizon.

Functional requirements

- System stores user preferences like passive/aggressive investing for each company or group of companies
- System provides for user a sentiment prediction
- System stores historical data for predictions and keeps them based on importance
- System pull email, api data directly from companies and other sources TBD
- Returns list of informations (verbal description that it based his decision on
- Lists crucial sources (chunks) for his decision

Non functional requirements

- Prompt has to execute in less than X minutes
- Model provides at least X% accuracy for request
- System stores previous predictions
- Model provides predictions up to two years period

Won't do

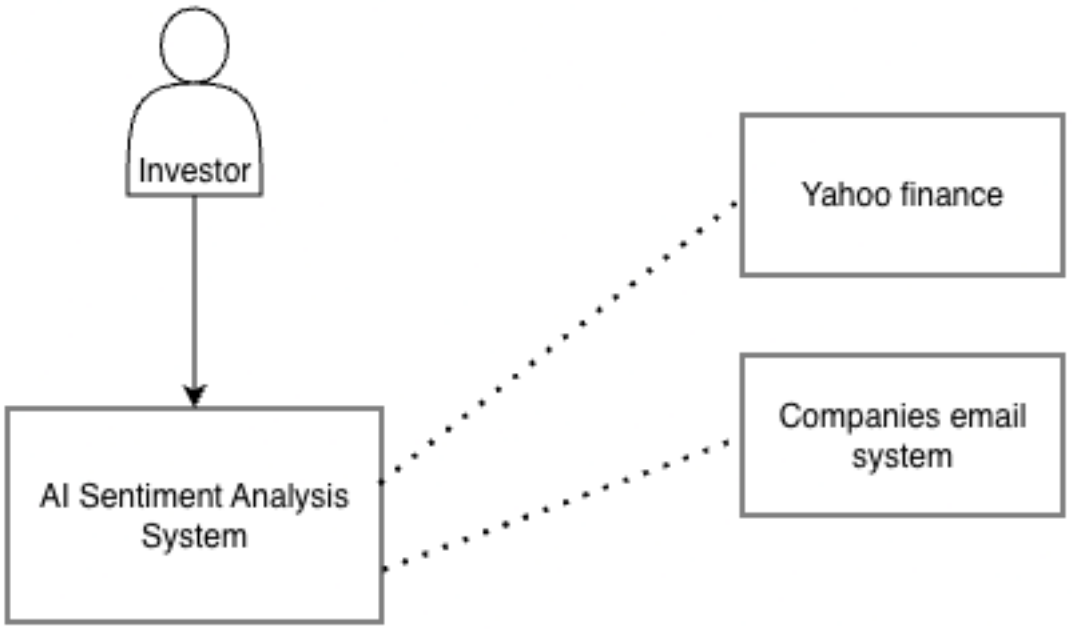
This section describes things that we won’t cover as part of this document and project

- Value based prediction, we only support positive/negative/neutral sentiment predictions
- Model supports historic data up to 3 years

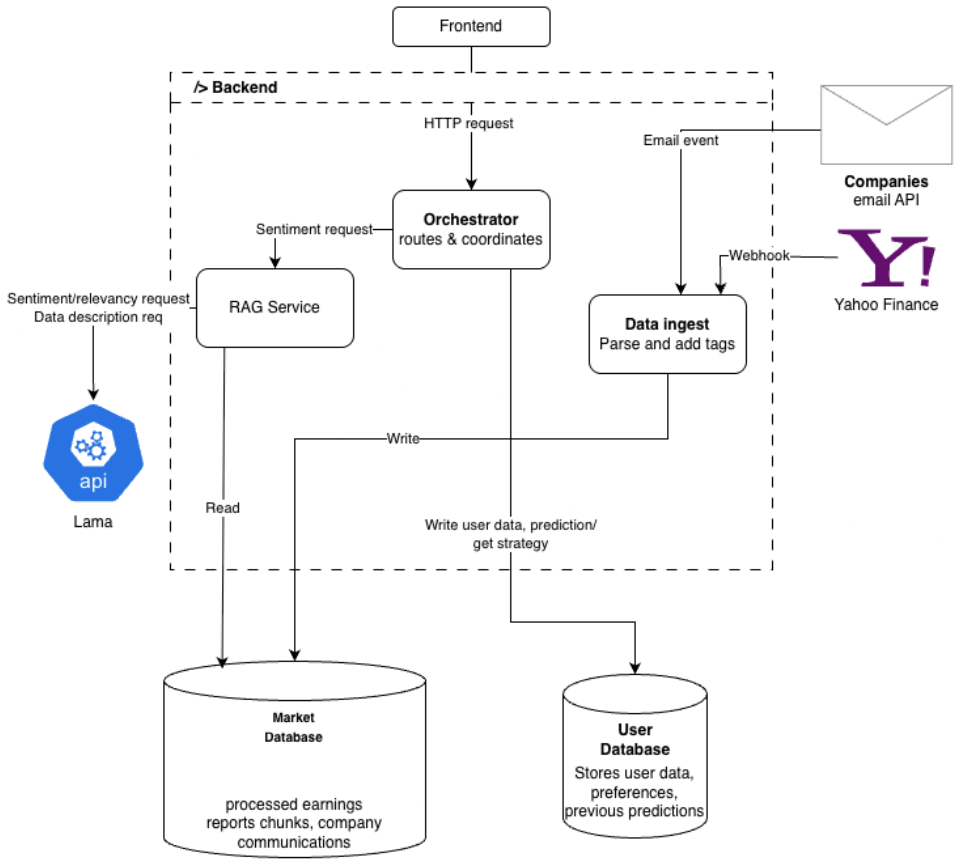
User stories

As a	I want	so that
Investor	get the sentiment prediction	I can improve my investing decision
Investor	characterize my investing strategy for company X	I get personalized suggestions
Admins	Have access to history of investors' requests	analyze and improve system predictions
Investor	to know based on what inf did the model predicted certain decision	I have possibility of verifying data myself

System components



Architecture diagram



Container diagram architecture

Backend endpoints

Endpoint	Method	URL	Headers	Body	Response
Writes user investing strategy for company X (may be list)	POST	`/user/strategy`	?	`{ strategy: Enum, desc: "text desc", companies: Enum[] }`	`{ "status": "success" }`
User creates account	POST	`/user/account`		`{ email: string,`	`{ "status": "success"`

				username: string }	}

Updates user investing strategy for company X (may be list)	PUT	/user/strategy	?	{
				strategy: Enum,
				desc: “text desc”,
				companies: Enum[]
				}
					{
					"status": "success"
					}

User request list of companies he currently is interested in (pre request for below)	GET	/user/strategy		None	{
					companies: Enum[]
					}

User request to see his current investing strategy for company X (may be list)	GET	/user/strategy
		{companyID}
				None	{
					strategies: dict{
					companyID : dict{
					strategy: Enum,
					desc: “text desc”
					}
					}
					}

Get prediction for company X	GET	/companies
		/{companyID}
		/prediction
			Authorization	None	{
					"companyID": 123,
					“sentiment”:
					"positive" | "negative",
					"confidence": float,
					“sources”: int[]
					}

Getting user requests history	GET	/user/history
		/{userID}
			?Admin?	None	{
					userID: 12,
					requests: {
					request_date: Date,
					question: String,
					prompt: String,
					response: String
					}
					}

Sending email from webhook	POST	/companies
		/{companyId}
			?Admin?	{
				email_content: “text”
				}
					{
					"status": "success"
					}

LLM Lama endpoints

Endpoint	Method	URL	Headers	Body	Response

Get relevancy assessment	POST	/lama/relevancy	Authorization	{
				timeframe: String,
				chunks_summaries: str[]
				}
					{
					chosen_chunks: int[]
					}

Get sentiment prediction	POST	/lama/prediction	Authorization	{
				company: Enum,
				timeframe: String,
				user_strategy: String,
				chunks: str[]
				}
					{
					sentiment: Enum,
					confidence: float 0-1,
					}

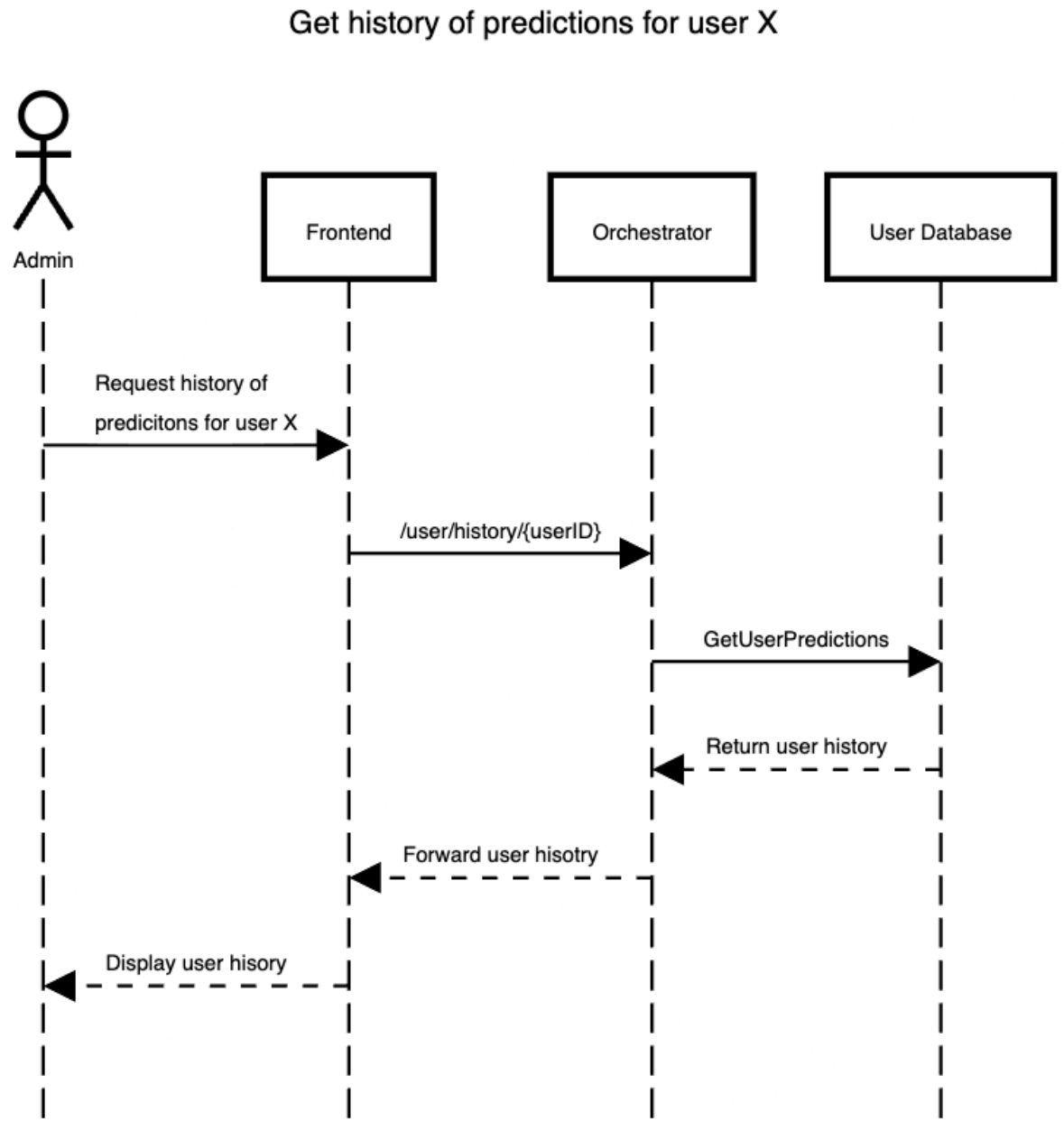
Market database operations

Operation	Description	Input	Output
GetEarningRaportEmbedding	Returns summaries of chunks of earning raptors and emails	`companyID`	`embedding[]`
GetEarningRaports	Return chunks of earning reports/ emails	`companyID, chunkID[]`	`chunks[]`
SaveMailContent	Saves received email	`companyID, string`	`status`

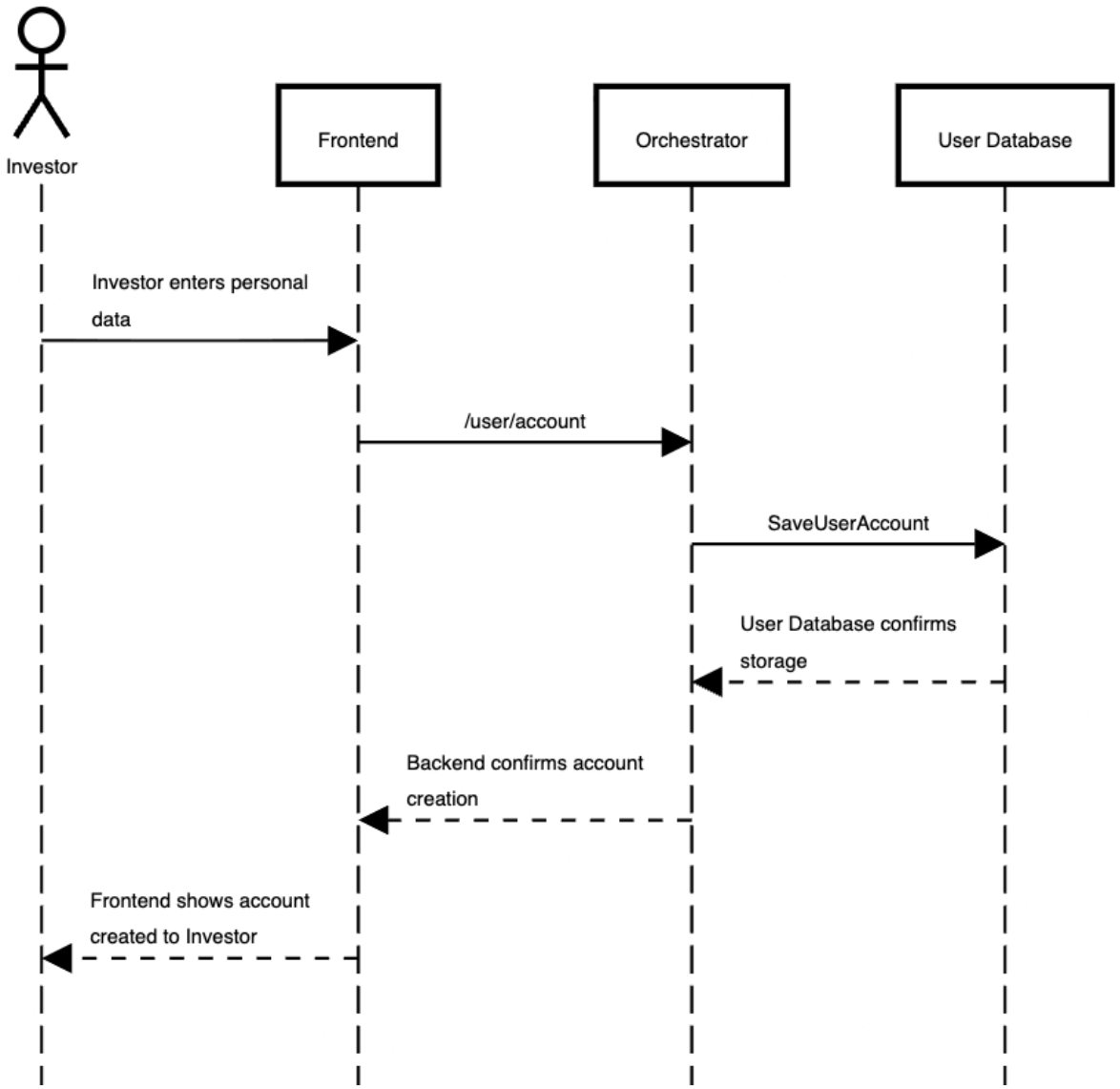
User database operations

Operation	Description	Input	Output
SaveUserStrategy	Saves user strategy	`userID, strategy`	`status`
GetUserStrategy	Retrieves user strategy	`userID,`	`strategy`
SaveUserPrediction	Saves prediction generated for user	`userID, strategy, prediction, date`	`status`
GetUserPredictions	Retrieves all user predictions	`userID`	`prediction[]`
SaveUserAccount	Saves user data after account creation	`userID, email, username`	`status`

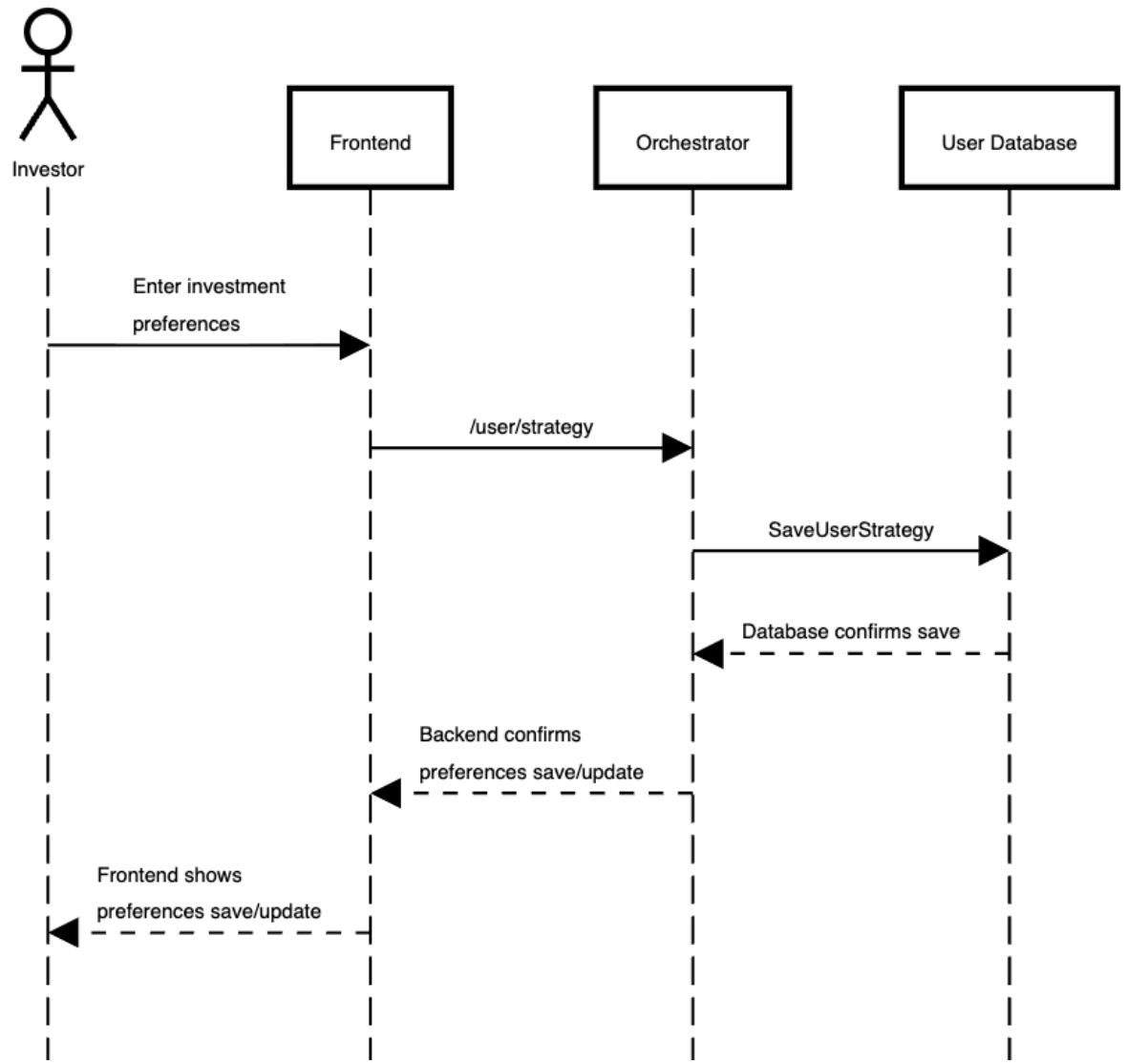
Sequence Diagrams



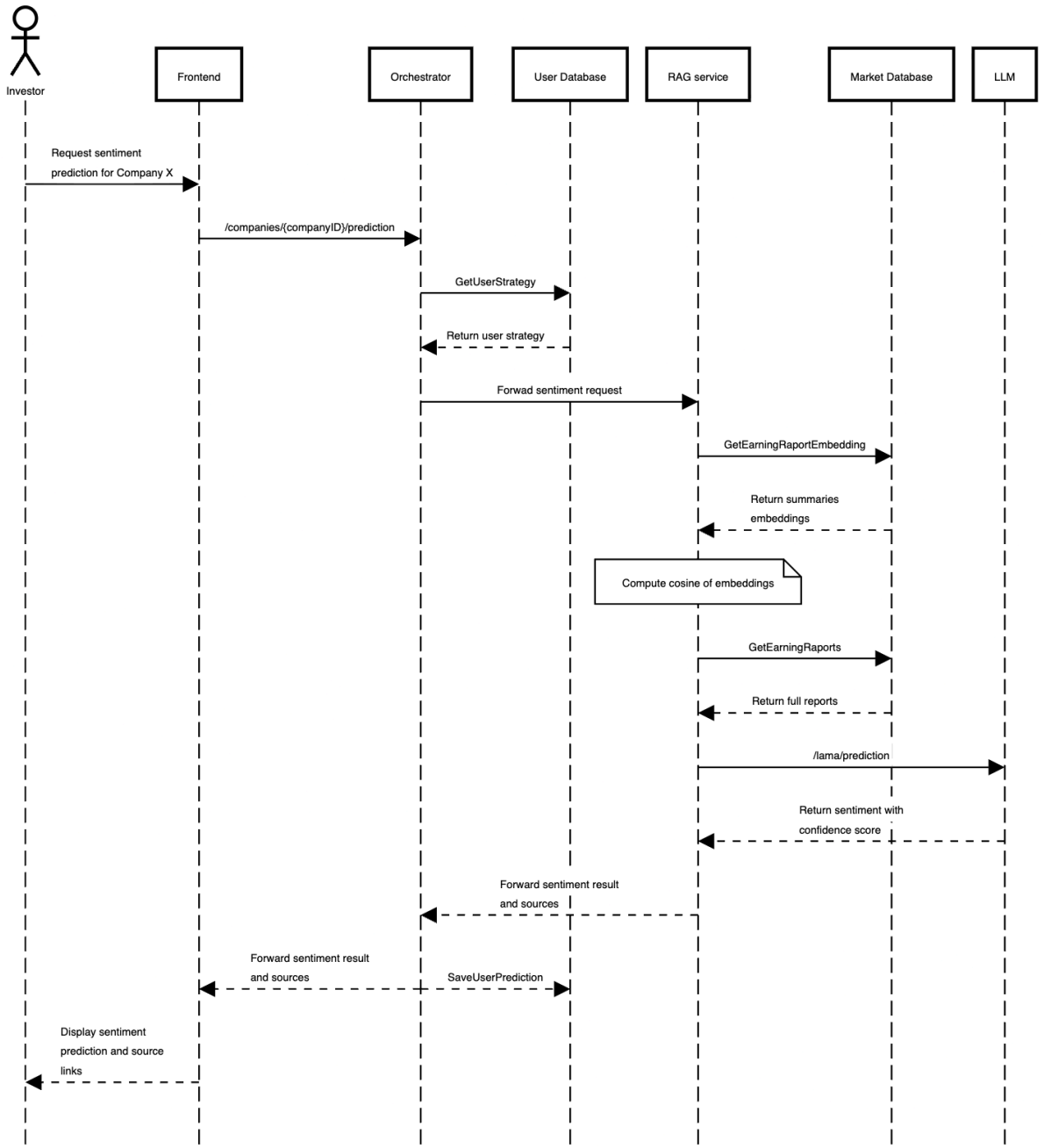
Investor account creation

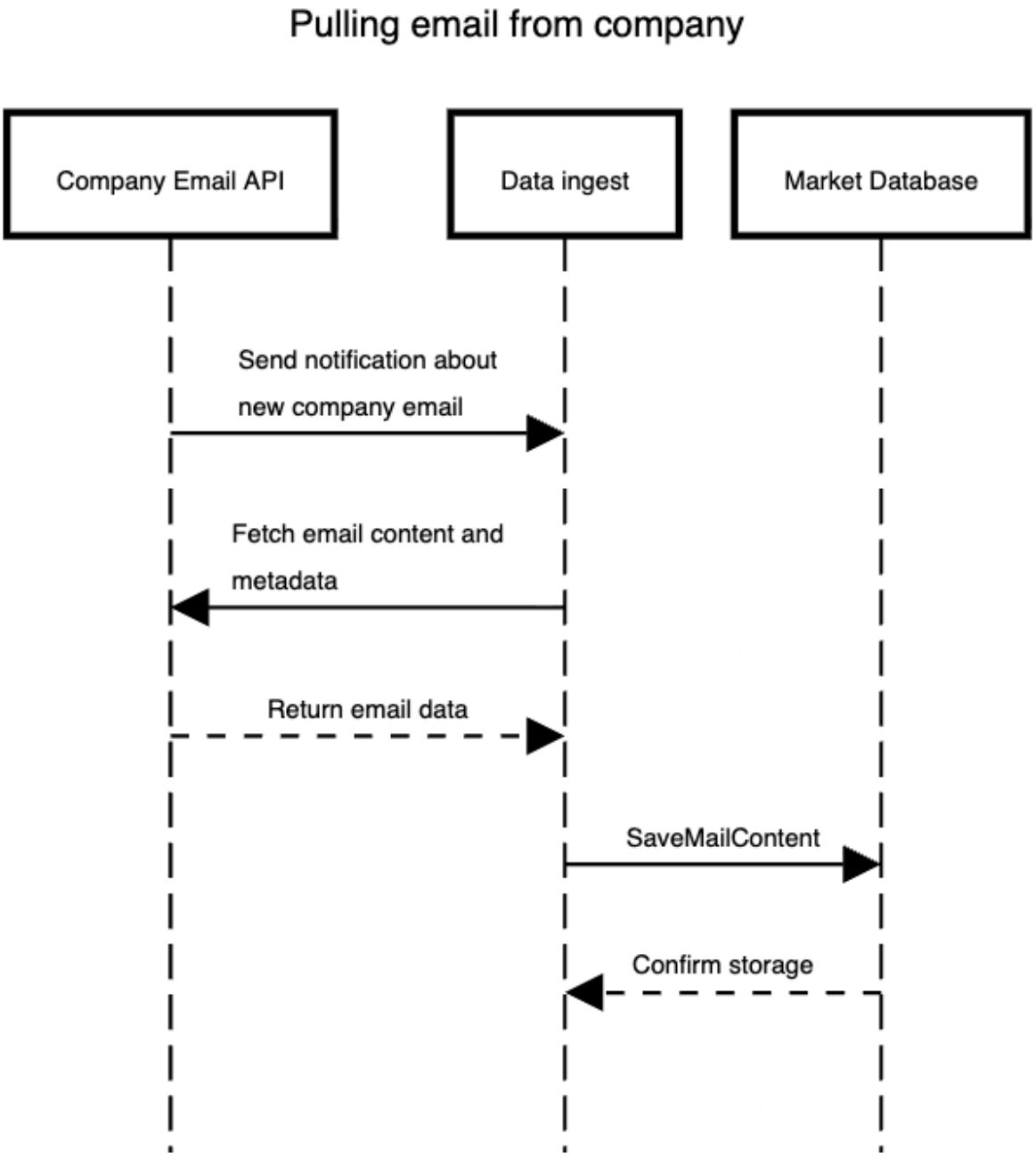


Investor updates/sets preferences



Get sentiment prediction




Pulling email from company
Company Email API
Data ingest
Market Database
Send notification about
new company email
Fetch email content and
metadata
Return email data
SaveMailContent
Confirm storage

Appendix